

Data Retrieval Repo

Data Retrieval

In this session, you'll set up your BigQuery environment and bring your data together. You'll download the Q1 2025 Cyclistic trip data, upload the monthly CSV files to BigQuery, and combine them into a single table using UNION ALL. By the end, you'll have one unified dataset ready for cleaning and analysis and you'll know exactly how many rides you're working with!

- Query 1: Preview data
- Query 2: Count individual tables
- Query 3: Count individual tables with alias
- Query 4: Create SSOT
- Query 5: Verify SSOT Count

Query 1: Preview data

```
SQL
SELECT *
FROM `CyclisticData2025.January2025`
LIMIT 10
```

Result/Observation: The first 10-rows of data from the January 2025 dataset.

Query 2: Count individual tables

```
SQL
SELECT COUNT(*) as table_rows
-- FROM `CyclisticData2025.January2025`
-- FROM `CyclisticData2025.Feburary2025`
FROM `CyclisticData2025.March2025`
```

Result/Observation:

January: 138,689

February: 151,880

March: 298,159

As the months get warmer, there are more cyclists.

Query 3: Count individual tables with an alias

SQL

Uncomment and comment, based on what table you are looking for.

```
SELECT COUNT(*) as table_rows  
FROM `CyclistData2025.January2025`
```

Expected: 138,689

```
-- FROM `CyclistData2025.Febuary2025`
```

Expected: 151,880

```
-- FROM `CyclistData2025.March2025`
```

Expected: 298,159

Query 4: Create SSOT

SQL

```
-- Create a new table, called q1_2025 that has all data from the 3
```

```
CREATE TABLE CyclistData2025.q1_2025 AS
```

```
SELECT * FROM `CyclistData2025.January2025`
```

```
UNION ALL
```

```
SELECT * FROM `CyclistData2025.Febuary2025`
```

```
UNION ALL
```

```
SELECT * FROM `CyclistData2025.March2025`
```

Result/Observation:

Created a table called q1_2025

▼	📅 CyclistData2025	☆	⋮
	📅 Feburary2025	☆	⋮
	📅 January2025	☆	⋮
	📅 March2025	☆	⋮
	📅 <u>q1_2025</u>	☆	⋮

Query 5: Verify SSOT count

SQL

```
SELECT COUNT(*) AS total_rows
FROM `CyclistData2025.q1_2025`
# Expected 588724
```

Result/Observation:

total_rows

588724:

Data Cleaning Repo

Data Cleaning

In this session, you'll investigate data quality issues and clean your dataset. You'll check for NULL values in critical fields, identify false starts and unreturned bikes using duration filters, and create a cleaned table that excludes invalid records. By the end, you'll have a trustworthy dataset with only analysis-ready rides!

- Query 6: Preview combined dataset.
 - Query 7: Establish baseline count.
 - Query 8a: Count member rides.
 - Query 8b: Count casual rides.
 - Query 9a: Check for missing user type.
 - Query 9b: Check for missing start times.
 - Query 9c: Check for missing end times.
 - Query 9d: Check for missing start station.
 - Query 9e: Check for missing end station.
 - Query 10: Preview rides with complete station data.
 - Query 11: Identify rides under 1 minute.
 - Query 12: Identify rides over 24 hours.
 - Query 13: Test combined filtering with AND.
 - Query 14: Alternative filtering with BETWEEN.
 - Query 15: Final combined preview, duration + station completeness.
 - Query 16: Create cleaned table with duration + station filters.
 - Query 17: Verify cleaned table row count
-

Query 6: Preview combined dataset

SQL

```
SELECT *  
FROM `CyclistData2025.q1_2025`  
LIMIT 10
```

Result/Observation:

First 10 rows of data from the q1_2025 table.

Query 7: Establish baseline count

```
SQL
SELECT COUNT(*)
FROM `CyclistData2025.q1_2025`
# Expected 588724
```

Result/Observation

total_rows

588724

Query 8a: Count member rides

```
SQL
SELECT COUNT(*) AS member_rides
FROM `CyclistData2025.q1_2025`
WHERE member_casual = "member"
# Expected: 450k~
```

Result/Observation:

member_rides

450948

Query 8b: Count casual rides

```
SQL
SELECT COUNT(*) AS casual_rides
FROM `CyclistData2025.q1_2025`
WHERE member_casual = "casual"
# Expected: 138k~
```

Result/Observation:

casual_rides

137776

Query 9a: Check for missing user type

```
SQL
SELECT COUNT(*) AS missing_member_casual
FROM `CyclistData2025.q1_2025`
WHERE member_casual IS NULL
# Expected: 0
```

Result/Observation:

missing_member_casual

0

Query 9b: Check for missing start times

```
SQL
SELECT COUNT(*) AS missing_started_at
FROM `CyclistData2025.q1_2025`
WHERE started_at IS NULL
# Expected: 0
```

Result/Observation:

missing_started_at

0

Query 9c: Check for missing end times

```
SQL
SELECT COUNT(*) AS missing_ended_at
FROM `CyclistData2025.q1_2025`
WHERE ended_at IS NULL
# Expected: 0
```


Result/Observation:

missing_ended_at
0

Query 9d: Check for missing start station

```
SQL
SELECT COUNT(*) AS missing_start_station
FROM `CyclistData2025.q1_2025`
WHERE start_station_name IS NULL
# NULLS Expected
```

Result/Observation:

missing_start_station
104117

Query 9e: Check for missing end station

```
SQL
SELECT COUNT(*) AS missing_end_station
FROM `CyclistData2025.q1_2025`
WHERE end_station_name IS NULL
# NULLS Expected
```

Result/Observation:

missing_end_station
109306

Query 10: Preview rides with complete station data

SQL

```
SELECT COUNT(*) AS rides_with_station
FROM `CyclistData2025.q1_2025`
WHERE end_station_name IS NOT NULL and start_station_name IS NOT NULL
# Expected: 400k~
```

Result/Observation:

rides_with_station

420649

Query 11: Identify rides under 1 minute

SQL

```
SELECT *
FROM `CyclistData2025.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) < 1
LIMIT 100;
```

Result/Observation:

There is a lot of data here, and it seems to check out when looking at the start/end times.

Query 12: Identify rides over 24 hours

SQL

```
SELECT *
FROM `CyclistData2025.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) > 1440
LIMIT 100;
```

Result/Observation:

Query 13: Test combined filtering with AND

SQL

```
SELECT COUNT(*) AS valid_ride_count
FROM `CyclistData2025.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) >= 1 AND
TIMESTAMP_DIFF(ended_at, started_at, MINUTE) <= 1440
```

Result/Observation:

valid_ride_count

576200

Query 14: Alternative filtering with BETWEEN

SQL

```
SELECT COUNT(*) AS valid_ride_count_alt
FROM `CyclistData2025.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) BETWEEN 1 AND 1440
#Expected: Same as Query 13, 576200
```

Result/Observation:

valid_ride_count_alt

576200

Query 15: Final combined preview, duration + station completeness

SQL

```
SELECT COUNT(*) AS valid_ride_count_alt
FROM `CyclistData2025.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) BETWEEN 1 AND 1440 AND
start_station_name IS NOT NULL AND end_station_name IS NOT NULL
# combining everything into one query
```

Result/Observation:

valid_ride_count_alt

417656

Query 16: Create a cleaned table with duration + station filters

SQL

```
CREATE OR REPLACE TABLE `CyclistData2025.q1_2025_cleaned` AS
SELECT *
FROM `CyclistData2025.q1_2025`
WHERE TIMESTAMP_DIFF(ended_at, started_at, MINUTE) BETWEEN 1 AND 1440 AND
start_station_name IS NOT NULL AND end_station_name IS NOT NULL
```

Result/Observation:

Created a new table with validated data points that are cleaned under the clauses we did throughout this data cleaning repo.

Query 17: Verify cleaned table row count

SQL

```
SELECT COUNT(*) as total_row
FROM `carbide-topic-492700-m0.CyclistData2025.q1_2025_cleaned`
#Expected: <420k~
```

Result/Observation:

total_row

417656

Data Transformation Repo

Data Transformation

In this session, you'll engineer new features from your cleaned data. You'll calculate ride duration in minutes, extract day of week and hour of day from timestamps, and categorize trips as round-trip or one-way using CASE statements. By the end, you'll have a processed dataset with 4 new columns that enable the pattern analysis to come!

- Query 18: Preview cleaned dataset with specific columns
- Query 19: Calculate ride duration (preview)
- Query 20: Extract day of week (preview)
- Query 21: Extract hour of day (preview)
- Query 22: Categorize trip type with CASE (preview)
- Query 23: Preview combined features
- Query 24: Create processed dataset (all features)
- Query 25: Verify row count
- Query 26: Preview processed features

Query 18: Preview cleaned dataset with specific columns

```
SQL
SELECT
    ride_id,
    started_at,
    ended_at,
    start_station_name,
    end_station_name,
    member_casual
FROM `CyclistData2025.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

The table outputs 7 columns, with what we are asking for, so its easier to read the data.

Example output:

Row	ride_id	started_at	ended_at	start_station_name	end_station_name	member_casual
1	6830B631 C00C0E22	2025-01-01 01:02:22.860000 UTC	2025-01-01 01:52:50.7020 00 UTC	Oconto Ave & Belmont Ave	Austin Blvd & Chicago Ave	casual

Query 19: Calculate ride duration (preview)

```
SQL
SELECT
    ride_id,
    started_at,
    ended_at,
    TIMESTAMP_DIFF(ended_at, started_at, MINUTE) AS ride_length_minutes
FROM `CyclistData2025.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

Row	ride_id	started_at	ended_at	ride_length_minutes
1	6830B631C00 C0E22	2025-01-01 01:02:22.860000 UTC	2025-01-01 01:52:50.702000 UTC	50

Query 20: Extract day of week (preview)

```
SQL
SELECT
    ride_id,
    started_at,
    EXTRACT(DAYOFWEEK FROM started_at) AS day_number
FROM `CyclistData2025.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

Row	ride_id	started_at	day_number
-----	---------	------------	------------

		2025-01-01	
	6830B631C00	01:02:22.86000	
1	C0E22	0 UTC	4

Query 21: Extract hour of day (preview)

```
SQL
SELECT
    ride_id,
    started_at,
    EXTRACT(HOUR FROM started_at) AS hour_number
FROM `CyclistData2025.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

Row	ride_id	started_at	hour_number
		2025-01-01	
	6830B631C00	01:02:22.86000	
1	C0E22	0 UTC	1

Query 22: Categorize trip type with CASE (preview)

```
SQL
SELECT
    ride_id,
    start_station_name,
    end_station_name,
    CASE
        WHEN start_station_name = end_station_name THEN 'Round Trip'
        ELSE 'One Way'
    END AS trip_type
FROM `CyclistData2025.q1_2025_cleaned`
LIMIT 10;
```


Result/Observation:

Row	ride_id	start_station_name	end_station_name	trip_type
1	6830B631C00C0E22	Oconto Ave & Belmont Ave	Austin Blvd & Chicago Ave	One Way

Query 23: Preview combined features

SQL

```
SELECT
    ride_id,
    member_casual,
    TIMESTAMP_DIFF(ended_at, started_at, MINUTE) AS ride_length_minutes,
    EXTRACT(DAYOFWEEK FROM started_at) AS day_number,
    CASE
        WHEN start_station_name = end_station_name THEN 'Round Trip'
        ELSE 'One Way'
    END AS trip_type
FROM `CyclistData2025.q1_2025_cleaned`
LIMIT 10;
```

Result/Observation:

Row	ride_id	member_casual	ride_length_minutes	day_number	trip_type
1	6830B631C00C0E22	casual	50	4	One Way

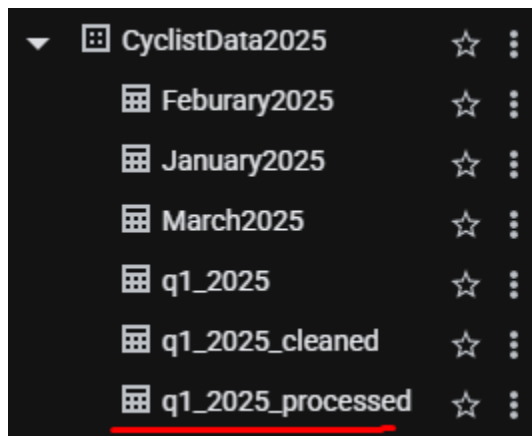
Query 24: Create processed dataset (all features)

SQL

```
CREATE OR REPLACE TABLE
`carbide-topic-492700-m0.CyclistData2025.q1_2025_processed` AS
SELECT *,
    TIMESTAMP_DIFF(ended_at, started_at, MINUTE) AS ride_length_minutes,
    EXTRACT(DAYOFWEEK FROM started_at) AS day_number,
```

```
EXTRACT(HOUR FROM started_at) AS hour_number,  
CASE  
    WHEN start_station_name = end_station_name THEN 'Round Trip'  
    ELSE 'One Way'  
END AS trip_type  
FROM `CyclistData2025.q1_2025_cleaned`
```

Result/Observation:



Query 25: Verify row count

```
SQL  
SELECT COUNT(*) as total_rows  
FROM `CyclistData2025.q1_2025_processed`  
# Expected 417k
```

Result/Observation:

total_rows
417656

Query 26: Preview processed features

```
SQL
SELECT
  ride_id,
  member_casual,
  ride_length_minutes,
  day_number,
  hour_number,
  trip_type
FROM `CyclistData2025.q1_2025_processed`
LIMIT 10
```

Result/Observation:

Row	ride_id	member_casual	ride_length_minutes	day_number	hour_number	trip_type
1	6830B631C00C0E22	casual	50	4	1	One Way

Data Aggregation Repo

Data Aggregation

In this session, you'll aggregate your processed data to reveal the patterns that answer Cyclistic's business question. You'll use GROUP BY, COUNT, AVG, and ORDER BY to compare members and casuals across temporal, duration, behavioral, and geographic dimensions. By the end, you'll have 9 insight tables ready for Tableau visualization and strategic recommendations!

- Query 27: Compare total rides by user type
- Query 28: Compare rides by day of week and user type
- Query 29: Compare rides by hour of day and user type
- Query 30: Calculate average ride length by user type
- Query 31: Count rides over 30 minutes by user type
- Query 32: Compare rides by trip type and user type
- Query 33: Compare rides by bike type and user type
- Query 34: Rank top 10 start stations for casuals
- Query 35: Rank top 10 end stations for casuals

Query 27: Compare total rides by user type

SQL

```
SELECT
    member_casual,
    COUNT(*) AS total_rides
FROM `CyclisticData2025.q1_2025_processed`
GROUP BY member_casual
```

Result/Observation:

<u>member_casual</u>	<u>total_rides</u>
casual	97504
member	320152

Query 28: Compare rides by day of week and user type

SQL

```
SELECT
```

```

    member_casual,
    day_number,
    COUNT(*) AS total_rides
FROM `CyclistData2025.q1_2025_processed`
GROUP BY member_casual, day_number
ORDER BY member_casual, day_number

```

Result/Observation:

<u>member_casual</u>	<u>day_number</u>	<u>total_rides</u>
casual	1	12041
casual	2	12499
casual	3	11164
casual	4	10506
casual	5	12034
casual	6	19484
casual	7	19776
member	1	29834
member	2	52040
member	3	49971
member	4	48593
member	5	53221
member	6	48989
member	7	37504

Query 29: Compare rides by hour of day and user type

```

SQL
SELECT
    member_casual,
    hour_number,
    COUNT(*) AS total_rides
FROM `CyclistData2025.q1_2025_processed`
GROUP BY member_casual, hour_number
ORDER BY member_casual, hour_number

```

Result/Observation:

<u>member</u>	<u>casual</u>	<u>hour</u>	<u>number</u>	<u>total</u>	<u>rides</u>
casual		0		1489	
casual		1		1087	
casual		2		662	
casual		3		426	
casual		4		394	
casual		5		581	
casual		6		1470	
casual		7		3037	
casual		8		4387	
casual		9		3787	
casual		10		3988	
casual		11		5373	
casual		12		6323	
casual		13		6767	
casual		14		7544	
casual		15		8488	
casual		16		9687	
casual		17		9907	
casual		18		7179	
casual		19		4689	
casual		20		3234	
casual		21		2751	
casual		22		2344	
casual		23		1910	
member		0		2010	
member		1		1163	
member		2		651	
member		3		503	
member		4		703	
member		5		3378	
member		6		10448	
member		7		20974	
member		8		26825	
member		9		16626	
member		10		13290	
member		11		15264	
member		12		17623	
member		13		17487	
member		14		17831	
member		15		22524	

member	16	32390
member	17	35190
member	18	23937
member	19	15249
member	20	9852
member	21	7654
member	22	5333
member	23	3247

Query 30: Calculate average ride length by user type

```
SQL
SELECT
    member_casual,
    ROUND(AVG(ride_length_minutes), 2) AS avg_ride_length
FROM `CyclistData2025.q1_2025_processed`
GROUP BY member_casual
```

Result/Observation:

<u>member_casual</u>	<u>avg_ride_length</u>
casual	16.93
member	10.03

Query 31: Count rides over 30 minutes by user type

```
SQL
SELECT
    member_casual,
    COUNT (*) as long_rides
FROM `CyclistData2025.q1_2025_processed`
WHERE ride_length_minutes > 30
GROUP BY member_casual
```

Result/Observation:

<u>member_casual</u>	<u>long_rides</u>
casual	11399
member	12109

Query 32: Compare rides by trip type and user type

```
SQL
SELECT
    member_casual,
    trip_type,
    COUNT (*) as long_rides
FROM `CyclistData2025.q1_2025_processed`
GROUP BY member_casual, trip_type
ORDER BY member_casual, trip_type
```

Result/Observation:

<u>member_casual</u>	<u>trip_type</u>	<u>long_rides</u>
casual	One Way	90472
casual	Round Trip	7032
member	One Way	314290
member	Round Trip	5862

Query 33: Compare rides by bike type and user type

```
SQL
SELECT
    member_casual,
    rideable_type,
    COUNT (*) as long_rides
FROM `CyclistData2025.q1_2025_processed`
GROUP BY member_casual, rideable_type
ORDER BY member_casual, rideable_type
```

Result/Observation:

<u>member_casual</u>	<u>rideable_type</u>	<u>long_rides</u>
casual	classic_bike	45183
casual	electric_bike	52321
member	classic_bike	167818
member	electric_bike	152334

Query 34: Rank top 10 start stations for casuals

```
SQL
SELECT
    start_station_name
FROM `CyclistData2025.q1_2025_processed`
WHERE member_casual = "casual"
GROUP BY start_station_name
ORDER BY COUNT(*) DESC
LIMIT 10;
```

Result/Observation:

start_station_name

Streeter Dr & Grand Ave
DuSable Lake Shore Dr & Monroe St
Shedd Aquarium
Millennium Park
Dusable Harbor
Kingsbury St & Kinzie St
Michigan Ave & Oak St
Canal St & Madison St
DuSable Lake Shore Dr & North Blvd
Clark St & Elm St

Query 35: Rank the top 10 end stations for casuals

```
SQL
SELECT
    end_station_name
FROM `CyclistData2025.q1_2025_processed`
WHERE member_casual = "casual"
GROUP BY end_station_name
ORDER BY COUNT(*) DESC
LIMIT 10;
```

Result/Observation:

end_station_name

Streeter Dr & Grand Ave
DuSable Lake Shore Dr & Monroe St
Millennium Park
Michigan Ave & Oak St
Shedd Aquarium
Dusable Harbor
Kingsbury St & Kinzie St
DuSable Lake Shore Dr & North Blvd
Canal St & Madison St
Clinton St & Washington Blvd
